Reproducing LLMLingua: Evaluating Accuracy–Efficiency Trade-offs in Long-Context Prompt Compression

CHEN IAT HONG MC654102
KONG KIN MENG MC652817
CHE CHON HONG MC551841
University of Macau
CISC7021 Applied Natural Language Processing – Course Project Final Report

Abstract

Long prompts improve retrieval-augmented and in-context question answering, but they also raise memory use, latency, and cost, and models may ignore information placed in the middle of a long context. Prompt compression attacks both problems without retraining the target model, yet its reported gains are usually measured with proprietary LLMs and reported as token reductions rather than end-to-end cost. We reproduce and evaluate four prompt settings on the full 150-example LongBench `multifieldqa_en` split with a 3B open-weight target model: an uncompressed baseline, head-and-tail truncation, LLMLingua, and a question-aware long-context variant of LLMLingua, the last two at nominal $2\times$ and $4\times$ compression. All settings share one fixed sample index, one prompt template, and one Exact Match / token-F1 implementation. Accuracy decreases monotonically with the compression ratio in every compressed family. Truncation keeps most of the baseline quality (F1 0.4065 and 0.3620 against 0.4296), whereas LLMLingua is markedly worse (0.2626 and 0.2207); replacing its chunked Phi-2 compressor with a single-pass question-aware compressor recovers part of the loss (0.3070 and 0.2794) but still falls short of truncation, the goal it was designed to reach. In a same-machine rerun of the first three settings, only truncation actually reduces end-to-end latency, by 44% and 66%, while LLMLingua at $2\times$ is 44% slower than the uncompressed baseline despite removing half of the prompt tokens. A position analysis of the truncation errors shows that the $4\times$ loss is mostly explained by more gold answers falling into the removed middle (21% to 37% of samples), not by worse answers on the samples whose evidence survives.

1 Introduction

1.1 Motivation

Large language models (LLMs) are now routinely asked to answer a question from several retrieved documents, to summarise a long article, or to learn a task from many in-context demonstrations. Each of these settings pushes thousands of tokens into a single prompt. The cost is not only monetary: a longer prompt occupies more GPU memory, needs more prefill computation, and therefore increases the time-to-first-token and the end-to-end latency of the serving system. Retrieved context is also repetitive. Documents overlap with one another and with the instruction, so a non-trivial fraction of the prompt carries little new information for the question at hand. Finally, a large advertised context window does not imply that the model uses it uniformly: Liu et al. (2024) show that accuracy is typically highest when the relevant passage sits at the beginning or the end of the context and drops when it sits in the middle.

Prompt compression addresses these issues by shortening the text that reaches the target model while trying to preserve the information the task needs. Unlike model compression or distillation, it requires neither retraining nor access to the target model's weights, so the same compressor can be placed in front of any open or proprietary endpoint. This makes it an attractive engineering option for resource-constrained deployments, and it is the reason a family of perplexity-based methods has been developed around it.

What is missing is not a new method but a clean measurement. The headline numbers in this literature are typically obtained with proprietary target models and are reported as reductions in input tokens. Token reduction, however, is only one side of the ledger: the compressor itself is a language model that must run on the same hardware, and its cost can exceed the generation time it saves. It is therefore unclear whether the accuracy–efficiency trade-off reported for large proprietary models still holds for a smaller open-weight model that a student project, or a small deployment, can actually afford to run—and whether the compressor pays for itself once its own latency is counted.

1.2 Literature Review

Li et al. (2025) organise the field and separate methods that delete or rewrite visible prompt tokens from methods that encode the prompt into learned continuous representations. The former are directly applicable to any frozen model and are the family studied here.

Selective Context (Li et al., 2023) removes lexical units whose self-information is low, showing that natural-language prompts contain substantial redundancy. Its importance estimate is not conditioned on the downstream question, so it may discard details that are predictable in general but essential for a specific query.

LLMLingua (Jiang et al., 2023) introduces a coarse-to-fine framework in which a small language model scores prompt components and individual tokens. A budget controller first allocates different compression budgets to instructions, demonstrations, and questions; iterative token-level compression then removes low-value tokens in several passes, and distribution alignment reduces the gap between the compressor and the target model. Reported gains reach roughly $20\times$ compression with limited quality loss on some tasks.

LongLLMLingua (Jiang et al., 2024) extends the idea to long-context question answering, where useful evidence is sparse and question-dependent. It adds question-aware document ranking, contrastive perplexity, dynamic budget allocation, document reordering, and subsequence recovery, and reports large token and cost savings on a multi-document Natural Questions variant (Kwiatkowski et al., 2019).

Two evaluation resources complete the picture. Liu et al. (2024) motivate the accuracy side of our measurement by documenting position bias, and LongBench (Bai et al., 2024) supplies the long-context question-answering data we use, in a format that requires no additional preprocessing.

1.3 Research Questions

We ask three questions about a small, reproducible configuration:

- **RQ1:** How does the compression ratio affect question-answering accuracy?
- **RQ2:** At the same input-token budget, does LLMLingua outperform simple truncation?
- **RQ3:** Does compression reduce end-to-end latency once the compression overhead is included?

1.4 Contributions

- A shared evaluation harness, used unchanged by all four experiments: one fixed index of the 150 official `multifieldqa_en` test examples, one chat-template prompt renderer, one set of Exact Match / token-F1 implementations, and one configuration module.
- Four compression settings implemented end to end and run to completion on the full split: uncompressed, head-and-tail truncation, LLMLingua, and a question-aware long-context variant of LLMLingua, the latter two at nominal $2\times$ and $4\times$ compression (seven result settings in total).
- A paired, per-sample comparison with 95% confidence intervals, which is more informative than comparing leaderboard averages and lets us state explicitly which differences survive noise.
- A single-machine rerun of the five uncompressed/truncation/LLMLingua settings, so that the latency question (RQ3) is answered on one GPU rather than by comparing four different machines.
- A negative result: LLMLingua loses to plain truncation at a comparable token budget, and Part 4, which was designed to overtake truncation, recovers only part of the gap (at most 31–42% of it can be attributed to chunking, because several factors changed at once) and still falls short.
- A token-offset answer-position analysis of the truncation baseline that localises where its errors come from, and a set of reproducible qualitative cases for LLMLingua.

2 Detailed Description

Our design follows one principle: every comparison must be attributable to the compression method and to nothing else. The project therefore separates a *frozen* evaluation contract (data, prompt wording, metrics, decoding parameters) from four *replaceable* compression components. Each component is a drop-in stage that maps a document set and a question to a (possibly shorter) prompt; everything downstream of that stage is shared. Seven result settings are produced: the uncompressed baseline, four compressed settings at two nominal ratios, and two variants of the LLMLingua family, all evaluated on the same 150 samples in the same order.

2.1 Task and Dataset

We use `multifieldqa_en` from LongBench (Bai et al., 2024), an English single-document question-answering task. Each example contains a question, one long article drawn from diverse sources (LaTeX papers, legal documents, government reports and web PDFs), and one or more gold answer strings. Three properties made it suitable for this project. First, it requires no preprocessing: the official JSONL already provides a single concatenated `context` field and a `question` field, so all four implementations consume exactly the same text. Second, its prompts are genuinely long: in our chat-templated form the mean prompt is 6,966.8 tokens and the longest is 15,026, which is long enough for compression to matter and short enough to run on a single 12–16 GB GPU. Third, the gold answers are short free-form strings, which the standard Exact Match and token-F1 implementations can score without a judge model.

We evaluate on the *complete* official test split of 150 examples rather than a subsample. A subsample would have introduced a second source of variance on top of the compression effect, and 150 examples already keep the standard error of a mean F1 near 0.024, which is small enough for the paired contrasts we report. The sample identifiers are stored once in `shared/samples.json` in a fixed order derived from seed 42, and every result file is checked against that list.

One property of the dataset matters for interpretation. Many gold answers are annotator-written sentences rather than verbatim spans of the context, so a verbatim search recovers the answer string in only 18 of the 150 samples. This inflates the difficulty of Exact Match (a model must reproduce the full sentence to score 1) and it is why our answer-position analysis in Section 3.6 relies on normalised matching plus a fuzzy fallback.

2.2 Models and Hardware

Target model. All generation uses Llama-3.2-3B-Instruct. The canonical identifier `meta-llama/Llama-3.2-3B-Instruct` is licence-gated on the Hugging Face Hub, so we load the weight-identical mirror `unsloth/Llama-3.2-3B-Instruct`. The model is natively a 128k-context model (`max_position_embeddings` = 131072 with Llama-3 RoPE scaling), so no sample in the split exceeds its window. We nevertheless impose a prompt budget of 32,768 tokens (32,672 after reserving 64 generation tokens and 32 tokens of margin). The cap exists only to bound the fp16 KV cache on a 12 GB card; the longest prompt uses less than half of it, and the recorded `truncated_samples` count is 0 for the baseline, which is what an "uncompressed" reference must be able to claim.

Compressors. Part 3 uses `microsoft/phi-2` (2.7B) as the LLMLingua compressor, the option documented by the official project. Part 4 replaces it with `Qwen2.5-1.5B-Instruct` loaded in 4-bit precision, which provides a much longer working context and supports question-conditioned compression. Because the compressor and the target model use different tokenizers, token counts are

always measured with one declared measurement tokenizer (`NousResearch/Llama-2-7b-hf`) and the achieved ratio is reported next to the nominal one.

Hardware. The four parts ran on four machines: a local RTX 3060 (Part 1), a Colab Tesla T4 in fp16 (Part 2), an Apple M5 Pro GPU through MPS (Part 3), and a Colab CUDA GPU (Part 4). This heterogeneity is acceptable for accuracy, which is deterministic up to greedy-decoding noise, but not for latency, because the same prompt takes very different times on these devices. We therefore added a fifth experiment that replays the five uncompressed/truncation/LLMLingua settings back to back on the RTX 3060 with a single environment, and every latency claim in this report comes from that rerun. Part 4 has no such counterpart, because its compressor could not be loaded on that host; its timings are reported separately and never compared with the rerun.

2.3 Shared Evaluation Harness

The harness is a small Python package, `shared/`, containing four modules: `samples.json` (the 150 examples in fixed order), `config.py` (model identifiers, token budgets, decoding parameters), `prompt_template.py` (prompt rendering and budget-aware construction) and `metrics.py` (Exact Match and token F1). Nothing in this package may be modified by an individual member; a change requires a group decision, because a single altered prompt delimiter or token budget silently invalidates all cross-method comparisons. Each member contributes a runner that consumes the package and writes one JSON record per sample.

2.3.1 Sample Index

The index is a list of 150 identifiers in a fixed order, generated once with seed 42 from the official split and frozen in version control. It does three things: it guarantees that all four methods answer the same questions, it fixes the iteration order so that resume logic and chunked schedules are reproducible, and it makes the paired statistical test possible, since a paired difference is only defined for two scores of the same item. We verified after every run that each result file contains exactly the 150 identifiers, in matching membership and without duplicates; all five per-part result files pass this check.

2.3.2 Prompt Template

Prompt wording follows the official LongBench template: a short system message ("answer questions based only on the provided text") and a user message of the form *Read the following text and answer the question briefly. / Text: …/ Question: …/ Answer:*. Prompts are built by applying the model's chat template with `add_generation_prompt=True` and are then tokenised with `add_special_tokens=False`, so the recorded `prompt_tokens` is the length of the exact byte sequence the model consumes, not the length of a proxy string. This matters for the ratio accounting: a compressor removes document tokens only, and the template overhead (system message, delimiters, question) is constant across settings.

For the uncompressed baseline we additionally implement a budget fallback: if a prompt exceeded 32,672 tokens, the document text would be truncated from the tail by binary search so that the instruction, the question and the `Answer:` suffix always survive, and the row would record `truncated=true` together with `original_prompt_tokens`. No sample in the index triggers it.

2.3.3 Metrics

We report Exact Match and token-level F1, both computed exactly as in the SQuAD evaluation script: prediction and gold are normalised (lower case, articles, punctuation and extra whitespace removed), EM is 1 only if the normalised strings are identical, and F1 is the harmonic mean of unigram precision and recall over the normalised token bags. Both are macro-averaged over the 150 samples.

We are explicit about what these metrics do not capture. F1 is lexical overlap, so a correct paraphrase can score 0.3 and a wrong answer that copies the question can score 0.4; EM is close to unusable for sentence-length gold answers, which is why the whole project reports EM only as a secondary number (the best setting reaches 0.0933, i.e. 14 of 150 exact matches). Generation is capped at 64 new tokens, so a long gold sentence is penalised even when the answer is semantically right. We deliberately did not add an LLM judge: it would have introduced a second model and broken the single-model contract that makes the four parts comparable.

2.3.4 Configuration

All runs use greedy decoding (`do_sample=False`), a temperature of 0, and `max_new_tokens` $= 64$; the prompt budget is $32{,}768 - 64 - 32 = 32{,}672$ tokens; generation runs in fp16 on GPU and fp16 on MPS. Every runner writes one JSON object per sample with the shared fields (`id`, `question`, `prompt_tokens`, `answer`, `em`, `f1`, `compression_latency`, `generation_latency`, `total_latency`) plus method-specific diagnostics (achieved ratio, chunk count, compressor settings). Records are appended to a JSONL checkpoint after every sample, so an interrupted run resumes without recomputation and a partially written result file is never treated as final.

2.4 Compared Methods

The two nominal ratios are defined on the document text. For a document of n tokens and a target ratio r, the budget is $b = \mathrm{round}(n/r)$ document tokens; the instruction and the question are never compressed. The *achieved* ratio is the measured one, reported as prompt-token reduction relative to the uncompressed baseline, and it is always smaller than the nominal ratio because the template and question overhead is fixed. We report the achieved ratio with every number, following the observation that tokenizer differences between compressor and target model make exact nominal ratios unattainable.

2.4.1 Part 1: Uncompressed Baseline

The baseline builds the prompt from the full concatenated context, with the budget fallback described above and no

compression stage (`compression_latency` is 0 by construction). It serves two roles: it is the quality reference for RQ1 and RQ2, and it is the cost reference for RQ3. Its mean prompt is 6,966.8 tokens, its F1 is 0.4296 and its EM is 0.0933. Because it uses the same renderer, tokenizer and scorer as every other part, no result can be dismissed as a harness artefact.

2.4.2 Part 2: Head-and-Tail Truncation

The truncation baseline keeps the first and last halves of the document budget (`head_frac` $= 0.5$) and deletes the middle. It is deliberately *non-semantic*: it never looks at the question and never scores a token, so it is the weakest reasonable competitor and the fairest lower bound on "what a practitioner would do in five minutes". The achieved document ratios are exactly 2.000 and 3.9999, and the mean prompts fall from 6,966.8 to 3,523.1 and 1,801.2 tokens (reductions of 49.4% and 74.1%). On the Colab T4 we additionally had to split the prefill into chunks (`--chunk-size` and `--chunk-threshold`) to avoid attention-matrix out-of-memory errors; the chunking is a serving detail and does not change the prompt. The kept head and tail tokens are decoded back to text and joined without a marker, so the model sees one continuous text with a silent cut in the middle. The `docs` field is empty in the result files because the runner stores the truncated text only when `--save-context` is set.

2.4.3 Part 3: LLMLingua

We reproduce LLMLingua with its documented coarse-to-fine pipeline: a budget controller, iterative token-level filtering with `iterative_size` $= 200$, and compression rates of 0.5 and 0.25 for the $2\times$ and $4\times$ targets. The compressor is `microsoft/phi-2` and the measurement tokenizer is Llama-2-7b, which avoids a tokenizer-file incompatibility between Llama 3.2 and the legacy Transformers release required by LLMLingua 0.2.2. Achieved ratios are 2.094 and 3.508, with mean prompts of 3,308.1 and 1,961.6 tokens. Because Phi-2 has a short working context, every document longer than 1,400 measurement tokens is split into ordered, non-overlapping chunks that are compressed independently and then concatenated; 148 of 150 rows per ratio are affected. This is a resource-constrained reproduction, not the whole-prompt LLMLingua of the paper, and it is recorded in every row and revisited in Section 3.3.

2.4.4 Part 4: LLMLingua-Enhanced

Part 4 was designed to improve LLMLingua enough to beat the Part 2 truncation baseline (plan targets: F1 ≈ 0.415 and 0.380 at $2\times$ and $4\times$), by testing the hypothesis that Part 3's deficit is caused by chunking. It replaces the Phi-2 compressor with a single-pass `Qwen2.5-1.5B-Instruct` compressor in 4-bit precision, enables question-aware compression (`condition_on_question=true`), ranks content with the LongLLMLingua method, uses a sentence-level filter, and forces one compression chunk (`max_num_compression_chunks` $= 1$). The result is that 0 of 150 rows per ratio are chunked, with mean prompts of 3,254.1 and 1,663.0 tokens (reductions of 53.3% and 76.1%). F1 rises to 0.3070 and 0.2794.

This design confounds several changes at once (compressor size, question awareness, sentence-level filtering, long context, 4-bit weights), so the improvement must be read as an *upper bound* on the chunking effect rather than as a clean ablation. Section 3.3 uses it exactly that way.

2.5 Implementation

Each part is a self-contained command-line runner with a shared record schema, which let the four members work in parallel from the same frozen index without waiting for each other. The pipeline for a compressed part has four stages: build the prompt from the index, run the compression stage, run generation and scoring, and append the row to a resumable checkpoint. Figure 1 sketches the data flow.

2.5.1 Two-Stage Compress and Evaluate Pipeline

For the LLMLingua settings the compressor and the target model never co-reside in memory. Stage one compresses every document and writes the compressed text to a JSONL checkpoint; the process then exits. Stage two loads only Llama-3.2-3B-Instruct, answers the questions and scores them. This halves peak memory and, more importantly, allows the two stages to use different `transformers` versions: LLMLingua 0.2.2 depends on a legacy model-cache interface, whereas Llama 3.2 requires a newer release. Because the stages are separate processes, the upgrade between them is safe and reproducible. Both stages append one record per item and resume automatically, which matters on Colab, where a session can be reclaimed.

2.5.2 Chunked Compression and Chunked Prefill

Two different chunking mechanisms appear in this project and should not be confused. *Compression chunking* (Part 3) splits a document into 1,400-token pieces because the compressor's context is short; it changes what the compressor sees and therefore the output, and it affected 296 of 300 result rows overall. *Prefill chunking* (Part 2) splits the forward pass of the target model into segments to bound attention memory; it does not change the prompt or the answer distribution, and is not expected to change any reported metric. Part 4 removes the first kind entirely (0 chunked rows), which is why it can be used to bound the chunking effect.

2.5.3 Memory and Out-of-Memory Handling

The Colab T4 has 16 GB and is shared with the display, so the practical budget is smaller. We handle out-of-memory errors at three levels: shortened compressor chunks, chunked prefill for the target model, and a skip-and-retry policy in Part 2 in which a failed sample is not written to the checkpoint, is recorded in `oom_skipped_ids`, and is retried when the same command is re-run; the prompt itself is never altered. No sample was dropped from the final result files: all rows in all five per-part files are present, which is a precondition for the paired analysis.

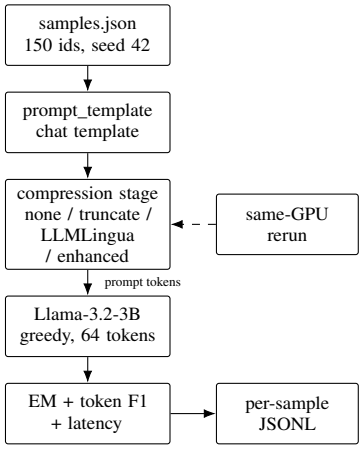


Figure 1: Data flow shared by all four parts. The frozen contract (sample index, template, decoding, metric) is drawn on the left; only the compression stage is replaced between settings. Dashed: the same-GPU rerun replays the five uncompressed/truncation/LLMLingua stages to make latency comparable.

2.5.4 System Overview

2.6 Experimental Setup

Seven result settings are reported, produced by five per-part runs plus one same-environment rerun (Table 1). Accuracy and token counts come from the per-part runs, which are the runs each member documented; latency comes from the rerun, which is the only run in which all five of its settings share one GPU and one software environment. Part 4 has no rerun counterpart.

2.7 Evaluation Protocol

F1 is the primary metric; EM is reported but is too strict to discriminate between settings on sentence-length gold answers. Because all methods answer the same items, we do not compare independent means. For each contrast we compute the per-sample difference $\Delta_i = F1_i^{\text{after}} - F1_i^{\text{before}}$, average it over the 150 items, and form a 95% confidence interval as $\bar{\Delta} \pm 1.96\,\mathrm{SE}(\Delta)$, where SE is the standard error of the paired differences. A contrast is called significant when this interval excludes zero. Pairing removes the between-question difficulty variance, which is large here (the per-setting standard deviation of F1 is about 0.29), and typically tightens the interval by a factor of two to three relative to an unpaired comparison. Latency is compared only within the rerun; per-part latency values are reported for completeness but never used in a claim, and Part 4's timings are quoted alone.

3 Results and Evaluation

3.1 Main Results

Table 2 reports the seven settings. Three patterns are visible immediately. First, F1 decreases monotonically with the compression ratio in all three compressed families: 0.4296 $\rightarrow$ 0.4065 $\rightarrow$ 0.3620 for truncation, 0.4296 $\rightarrow$ 0.2626 $\rightarrow$ 0.2207 for LLMLingua, and 0.4296 $\rightarrow$ 0.3070 $\rightarrow$ 0.2794 for the enhanced variant. Second, the size of that decrease differs enormously between families: at $2\times$ truncation loses 5.4% of relative F1 while LLMLingua loses 38.9%. Third, Exact Match collapses much faster than F1 and is essentially uninformative for the LLMLingua settings (0 to 2 exact matches of 150) and low for truncation (10 to 11 of 150), which is why we treat F1 as the measurement and EM as a sanity check.

3.2 RQ1: Accuracy versus Compression Ratio

Figure 2 plots F1 against the achieved token reduction. The answer to RQ1 is yes, accuracy falls with the ratio, and it falls monotonically inside every family. The magnitudes, however, are not comparable. Going from no compression to $4\times$ costs 0.0676 F1 with truncation but 0.2090 with LLMLingua, a threefold difference at the same nominal budget.

The paired contrasts (Table 3) sharpen this. The truncation loss at $2\times$ is small enough to be indistinguishable from noise (-0.0231, $[-0.057, +0.011]$), so no reliable loss is detectable in the per-part runs (the same-machine rerun shows a marginal loss; see Threats to Validity); at $4\times$ the loss becomes real (-0.0676) but is still modest. Both LLMLingua settings lose far more, and the enhanced variant loses less than LLMLingua while remaining clearly below the baseline. Notably, the per-sample spread of F1 is almost unchanged across settings (0.24–0.30), so compression shifts the whole distribution down rather than creating a few catastrophic failures.

3.3 RQ2: LLMLingua versus Truncation at Equal Token Budget

No. At equal budget, every LLMLingua variant is significantly worse than head-and-tail truncation. At $2\times$ the gap is -0.1440 $[-0.186, -0.102]$, and at $4\times$ it is -0.1414 $[-0.183, -0.100]$; both intervals are far from zero, so this is not a small-sample artefact. The budgets are close but not identical: at $4\times$ LLMLingua keeps 1,962 prompt tokens against 1,801 for truncation (about 9% more), so its loss is not caused by a tighter budget; at $2\times$ it keeps 3,308 against 3,523 (about 6% fewer); the enhanced variant keeps 3,254 and 1,663.

Part 4 exists to test the most obvious explanation. Part 3 had to compress 296 of 300 rows in chunks because Phi-2 cannot hold a full LongBench document, which makes it a different algorithm from the paper's whole-prompt version. Part 4 removes chunking completely (0 of 300 rows) by using a long-context compressor, and it does improve: $+0.0444$ $[+0.013, +0.076]$ at $2\times$ and $+0.0587$ $[+0.025, +0.093]$ at $4\times$ over Part 3, both significant. But the improvement recovers only about 31% of the $2\times$ gap and 42% of the $4\times$ gap, and the enhanced variant remains significantly below truncation (-0.0995 and -0.0827). The chunking hypothesis is therefore *partially* supported: it plausibly contributes, but at most 31–42% of the gap can be attributed to it, and a question-aware, de-chunked, long-context compressor still does not beat deleting the middle of a document.

In short, Part 4 did not reach its design goal of overtaking truncation: it stays 0.0995 and 0.0827 F1 below Part 2 and about 0.108 and 0.101 below its own targets. Two caveats belong with this conclusion. Part 4 changes five factors at once (compressor size, question conditioning, sentence-level filtering, context length, 4-bit weights), so its gain is an upper bound on the chunking effect, not a measurement of it. And the targets recorded in the project plan (F1 $\approx$ 0.415 and 0.380) were not met, so it is a negative result for *this* implementation of the paper's claim, not evidence that LLMLingua cannot work.

Setting	Device	Achieved ratio	Result file
Uncompressed	RTX 3060	1.000	`part1/results_member1.json`
Head-tail 2x	Colab T4	2.000	`part2/results_member2_2x.json`
Head-tail 4x	Colab T4	3.9999	`part2/results_member2_4x.json`
LLMLingua 2x	M5 Pro (MPS)	2.094	`part3/results_member3.json`
LLMLingua 4x	M5 Pro (MPS)	3.508	`part3/results_member3.json`
LLMLingua-Enh. 2x	Colab CUDA	2.14†	`part4/results_member4.json`
LLMLingua-Enh. 4x	Colab CUDA	4.19†	`part4/results_member4.json`
Rerun (5 settings)	RTX 3060	as above	`results_same_machine/`

Table 1: Experimental setup. Ratios are document-level achieved ratios.
†Whole-prompt reduction measured directly: 53.3% and 76.1%.

Method	n	EM	F1	F1 sd	Mean prompt tokens	Token reduction	Compress. s	Total s
Uncompressed	150	0.0933	**0.4296**	0.2910	6,966.8	—	0.000	4.65
Head-tail 2×	150	0.0733	0.4065	0.2931	3,523.1	49.4%	0.019	5.82
Head-tail 4×	150	0.0667	0.3620	0.2978	1,801.2	74.1%	0.018	3.55
LLMLingua 2×	150	0.0067	0.2626	0.2675	3,308.1	52.5%	10.816	13.84
LLMLingua 4×	150	0.0133	0.2207	0.2396	1,961.6	71.8%	9.846	11.98
LLMLingua-Enh. 2×	150	0.0000	0.3070	0.2571	3,254.1	53.3%	16.913	19.64
LLMLingua-Enh. 4×	150	0.0067	0.2794	0.2559	1,663.0	76.1%	14.654	16.48

Table 2: Main results, all seven settings, per-part runs. Accuracy and token counts are the numbers used in all claims. The two latency columns come from four different machines and are shown for completeness only; comparable latencies are in Table 4.

Contrast	$\overline{\Delta F1}$	95% CI	Verdict
Uncomp. → HT 2×	−0.0231	[−0.057, +0.011]	n.s.
Uncomp. → HT 4×	−0.0676	[−0.107, −0.028]	sig.
Uncomp. → LL 2×	−0.1670	[−0.208, −0.127]	sig.
Uncomp. → LL 4×	−0.2090	[−0.250, −0.168]	sig.
Uncomp. → Enh. 2×	−0.1226	[−0.162, −0.083]	sig.
Uncomp. → Enh. 4×	−0.1503	[−0.192, −0.108]	sig.
HT 2× → LL 2×	−0.1440	[−0.186, −0.102]	sig.
HT 4× → LL 4×	−0.1414	[−0.183, −0.100]	sig.
LL 2× → Enh. 2×	+0.0444	[+0.013, +0.076]	**gain**
LL 4× → Enh. 4×	+0.0587	[+0.025, +0.093]	**gain**
HT 2× → Enh. 2×	−0.0995	[−0.142, −0.057]	sig.
HT 4× → Enh. 4×	−0.0827	[−0.128, −0.037]	sig.

Table 3: Paired per-sample F1 contrasts on the same 150 items (per-part runs). HT = head-and-tail truncation, LL = LLMLingua, Enh. = LLMLingua-Enhanced, n.s. = interval includes zero.

3.4 RQ3: End-to-End Latency

Table 4 gives the only latencies we compare, all from one RTX 3060 with one software stack. Truncation reduces total latency by 43.6% at 2× and 65.7% at 4×, because it removes tokens and costs nothing: its compression stage takes 0.02 s. LLMLingua at 2× is 43.9% *slower* than the uncompressed baseline (13.57 s against 9.43 s): generation time does fall from 9.40 s to 6.03 s, but compression adds 7.50 s, more than the saving. At 4× the picture improves slightly (10.76 s total) but the setting is still slower than doing nothing.

Part 4 is worse still: its compression stage alone costs 16.9 s and 14.7 s per sample, so its total is 19.6 s and 16.5 s on the machine that ran it, i.e. several times the generation time alone. Since Part 4 has no same-hardware counterpart, we do not convert this into a ratio against the uncompressed baseline; the qualitative conclusion is nevertheless safe, because a compression stage that costs many times the generation time (14.7–16.9 s against 1.8–2.7 s) cannot be a latency optimisation under any plausible hardware assumption. The answer to RQ3 is thus: compression reduces end-to-end latency only when the compression step is free, and LLMLingua's inference cost is not free.

Method	n	F1	Prompt	Gen. s	Comp. s	Total s
Uncompressed	150	0.4350	6,966.8	9.403	—	9.43
Head-tail 2×	150	0.3990	3,523.1	5.291	0.020	5.32
Head-tail 4×	150	0.3633	1,801.2	3.209	0.019	3.23
LLMLingua 2×	150	0.2569	3,309.1	6.032	7.496	13.57
LLMLingua 4×	150	0.2221	1,962.8	4.034	6.711	10.76

Table 4: Same-environment rerun on a single RTX 3060 (`run_all.py`). All latency claims in this report come from this table. Part 4 is excluded because its compressor could not be loaded on this host.

3.5 Qualitative Analysis

Four paired cases from the committed result files explain the aggregate numbers; they are regenerated by a script with explicit selection rules, so they are not cherry-picked by hand.

Case 1 — lossless compression of a salient fact. Asked when *Weep Not, Child* was first published, the gold answer appears at 0.2% of the document and is repeated later. LLMLingua answers correctly at both ratios (F1 1.000), as does the baseline. When the fact is short, salient and redundant, token-level deletion costs nothing.

Case 2 — compression succeeds where position fails. For a question about the infall rate and gas density in a magnetised accretion model, the evidence sits at 65.1% of the document, outside both kept regions. Truncation at 2× and 4× collapses to F1 0.182 and answers that "the text does not provide a direct comparison"; LLMLingua keeps the numerical relation (F1 0.667 and 0.788). This is the mechanism the LLMLingua papers promise, and it is real.

Case 3 — a precise detail is deleted. Asked for a publication date, truncation answers "7 March 2023" correctly at both ratios, LLMLingua at 2× drops the year (F1 0.923)

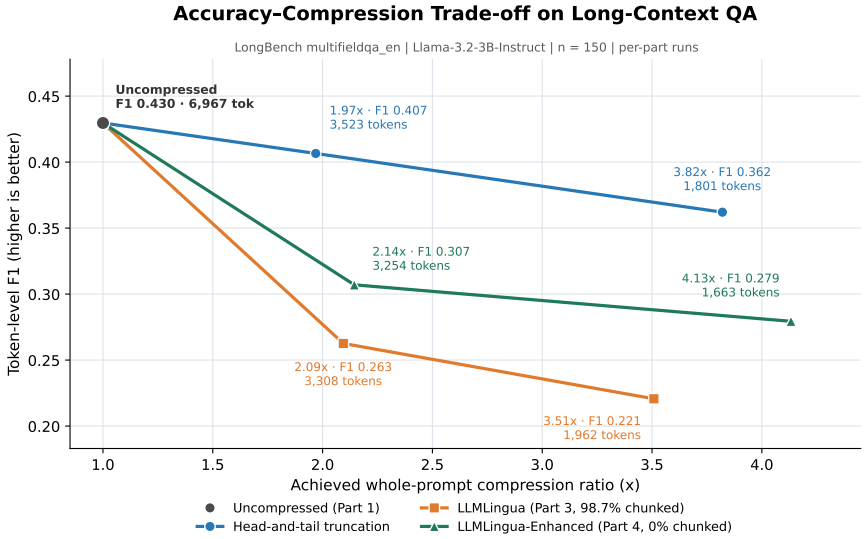


Figure 2: F1 against achieved prompt-token reduction for all seven settings. Truncation stays close to the uncompressed baseline at both ratios; the LLMLingua family degrades sharply, and the enhanced variant sits between the two but below truncation at equal budget.

and at 4× answers that the date is not mentioned (F1 0.133). The date is near the start of the document, so this is a pure compression error, not a positional one.

Case 4 — a plausible distractor survives. Asked in which electorate Simon English was elected, the baseline and truncation answer "Wallace"; LLMLingua answers "Clutha-Southland" at both ratios. The source contains both names, separated by a relation the compression removed. Token deletion preserves entities and can destroy the relation that makes one of them correct.

The cases jointly explain why the average gap is large while individual examples can favour compression: LLMLingua removes redundancy well and relations poorly.

3.6 Answer-Position Analysis of the Truncation Baseline

Truncation deletes the middle of the document, so its errors should concentrate where the evidence is. To test this we locate each gold answer in the *original* document and map it to token offsets with the tokenizer's offset mapping, then compare the span with the kept head and tail. A sample is `kept_head` or `kept_tail` when an occurrence of the answer lies fully inside the kept region, `partial` when an occurrence overlaps a boundary, `dropped_middle` when every occurrence falls in the removed middle, and `not_found` when no occurrence can be located (yes/no answers, inferred answers). Matching is normalised exact matching first; when that fails we take the best sliding window by content-word overlap (threshold 0.6) and mark the row `fuzzy`. This yields 46 exact, 87 fuzzy and 17 unmatched locations, identical at both ratios because the location does not depend on the ratio. The fuzzy locations are approximate, so all position results are correlational.

Tables 5–6 and Figures 3–5 give three findings. First, F1 is clearly lower when the gold answer falls in the removed middle: 0.226 against 0.474 at 2×, and 0.201 against 0.491 at 4×, with non-overlapping bootstrap intervals. Second, the *share* of affected samples, not the within-group quality, drives the ratio effect: the dropped-middle group grows from 31 to 56 of 150 (21% to 37%) when moving from 2× to 4×,

Ratio	Zone	n	Mean F1	Mean EM
2×	kept_head	85	0.4921	0.1176
2×	kept_tail	16	0.3768	0.0000
2×	partial	1	0.2069	0.0000
2×	dropped_middle	31	0.2259	0.0323
2×	not_found	17	0.3481	0.0000
4×	kept_head	63	0.5109	0.1587
4×	kept_tail	11	0.3762	0.0000
4×	partial	3	0.3501	0.0000
4×	dropped_middle	56	0.2006	0.0000
4×	not_found	17	0.3351	0.0000

Table 5: Answer-position analysis of the truncation baseline: mean F1 by zone. Group sizes are reported with every number because `partial` and `kept_tail` are small.

Ratio	Pooled zone	n	Mean F1	95% CI
2×	Kept (head+tail)	101	0.474	[0.416, 0.531]
2×	Partial	1	0.207	—
2×	Dropped	31	0.226	[0.154, 0.311]
4×	Kept (head+tail)	74	0.491	[0.419, 0.564]
4×	Partial	3	0.350	[0.205, 0.429]
4×	Dropped	56	0.201	[0.149, 0.255]

Table 6: Pooled zones with 95% bootstrap confidence intervals (`answer_position_stats`). Intervals for Kept and Dropped do not overlap at either ratio.

while F1 inside the kept group is almost unchanged (0.474 → 0.491) and F1 inside the dropped group moves only from 0.226 to 0.201. Third, answers cluster near the start of the document (`kept_head` is the largest zone, 85 of 150 at 2×), which is exactly why a head-and-tail heuristic remains competitive: most of the time it keeps the right region by construction.

The comparison with the uncompressed baseline is informative in the other direction. F1 on kept answers (0.47–0.49) is at or slightly above the baseline mean (0.4296), so on the samples truncation does not damage, quality is close to the reference; this compares a subset with the full-sample mean, so it is indicative only; the uncompressed F1 on the same

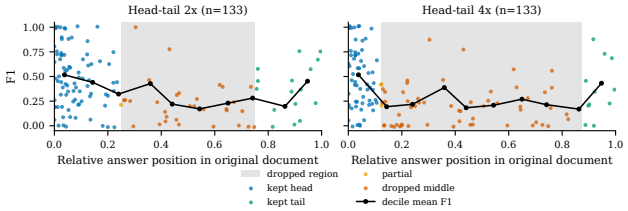


Figure 3: F1 against the relative position of the gold answer in the original document. The removed middle region is shaded; decile means trace the same pattern as the zone analysis.

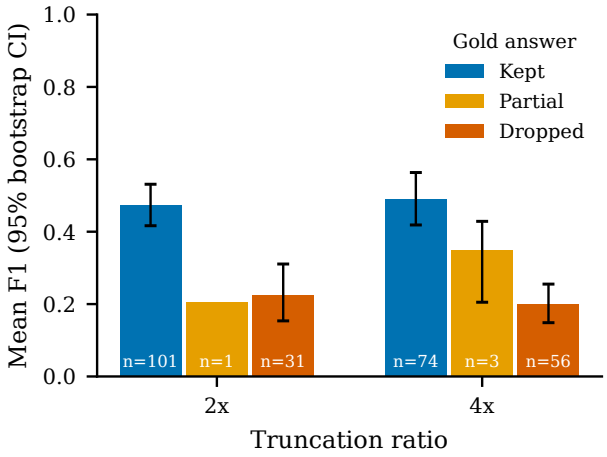


Figure 4: Mean F1 by pooled answer zone at $2\times$ and $4\times$. F1 on kept answers is essentially flat and close to the uncompressed baseline; the loss is concentrated in dropped answers.

subset would be the fairer reference. The residual error on kept answers therefore comes from the generator and from lexical F1, not from truncation. This is consistent with the insignificant $2\times$ contrast in Table 3.

3.7 Discussion

Why does a perplexity-based compressor lose to a baseline that knows nothing about the question? We offer four hypotheses that are consistent with our data but were not tested directly.

Fragmentation. Token-level deletion removes words whose loss is locally predictable, which is not the same as removing words whose loss is harmless for a downstream answer. Relations expressed as "the seat was later renamed Clutha-Southland" survive entity deletion as two unrelated entities (Case 4), while short high-information tokens such as years and numbers disappear (Case 3). Truncation is coarse but syntactically coherent: what it keeps, it keeps whole.

Target-model sensitivity. LLMLingua's published numbers use much larger target models. A 3B model is more sensitive to the grammatical and referential damage left by token deletion: the compressed prompt is no longer fluent, and a small model has less capacity to reconstruct the missing relation. Our results are therefore a statement about small open-weight deployments, and they do not refute the paper's results at 7B–70B scale.

Prompt geometry. In our setting the question follows the context and is never compressed, and the context is one long article whose evidence is not uniformly spread. Both favour the positional baseline: evidence tends to appear early (85 of

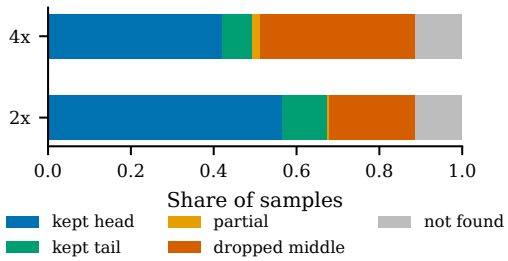


Figure 5: Share of samples in each zone. The dropped-middle share rises from 21% to 37% between $2\times$ and $4\times$, which is the main driver of the $4\times$ loss.

150 answers in the head zone), so a degenerate policy keeps the right region most of the time. In a setting where evidence is uniformly spread, the positional baseline would degrade and the gap would shrink.

Cost accounting. The compression stage is a forward pass of a 1.5–2.7B model over the whole document, which is comparable to the generation it is meant to accelerate. Even a large token reduction therefore cannot pay for itself at these prompt lengths; it can only do so when prompts are much longer than the compressor's own cost, or when the compressor is shared across many queries.

3.8 Threats to Validity

Hardware heterogeneity. Accuracy is deterministic up to decoding noise, but we observed that 12 of 150 uncompressed answers differ word for word between two GPUs under greedy decoding, so small F1 differences between runs are not meaningful. All latency claims are restricted to the single-GPU rerun.

Significance flips. The uncompressed $\rightarrow$ head-and-tail $2\times$ contrast is insignificant in the per-part runs (-0.0231, $[-0.057, +0.011]$) and barely significant in the rerun (-0.0360, $[-0.069, -0.003]$). We report it as a marginal loss at the edge of significance rather than choosing the friendlier run.

Stale export. The committed `results_table.md` inside `results_same_machine` was generated from the per-part result files rather than from the rerun files stored beside it, so it still shows cross-machine latencies; the authoritative rerun numbers are the ones in Table 4, regenerated directly from the rerun JSON files.

Single dataset, model, and hyper-parameter. One task, one target model, one head fraction, two ratios. The "truncation beats compression" ranking is established for `multifieldqa_en` with Llama-3.2-3B and should not be generalised to other benchmarks without replication.

Part 4 is not a clean ablation. Its gain is an upper bound on the chunking effect; it also has no common-hardware latency measurement, so no cross-method latency claim involving Part 4 is made.

Interface deviation. The `docs` field is empty in all five result files, deviating from the interface declared in the plan. This affects all parts equally, so comparability is preserved, but it prevents later re-scoring of compressed prompts without rerunning the pipelines.

Position analysis. 87 of 150 answer locations come from fuzzy matching, 17 cannot be located, and the analysis is correlational: questions whose evidence appears early may

also be intrinsically easier. Group sizes for `partial` and `kept_tail` are too small to support separate claims.

4 Conclusion

4.1 Summary of Findings

RQ1. Accuracy falls monotonically with the compression ratio in every family, but the price differs by a factor of three: at $4\times$, truncation loses 0.0676 F1 while LLMLingua loses 0.2090. **RQ2.** At an equal token budget LLMLingua never beats truncation: it loses 0.1440 and 0.1414 F1, and a de-chunked, question-aware variant recovers only 0.0444 and 0.0587 of that gap while remaining 0.0995 and 0.0827 below truncation. **RQ3.** Compression reduces end-to-end latency only when the compressor is cheap: truncation cuts total latency by 43.6% and 65.7%, whereas LLMLingua at $2\times$ is 43.9% slower than doing nothing because compression costs 7.5 s per sample.

4.2 Lessons Learned

Three lessons generalise beyond this project. First, *an evaluation contract is a research instrument*: freezing the sample index, the prompt template and the metric cost us a day of engineering and bought us the ability to compare four independently developed pipelines without a single re-run. Second, *token reduction is not efficiency*: the headline number in the compression literature is an input-side quantity, and the only defensible efficiency claim is measured end-to-end on one machine. Third, *a negative result needs a diagnosis to be useful*: "LLMLingua did worse" would have been uninterpretable, whereas "it did worse, chunking can explain at most 31–42% of it, and the rest is untested (we hypothesise fragmentation)" is a finding that constrains the next experiment.

4.3 Problems and Difficulties

Gated model access. The canonical Llama-3.2 identifier requires accepting a licence; we standardised on the weight-identical ungated mirror, and verified identity by comparing model cards rather than assuming it.

Windows console encoding. One baseline run crashed at sample 87 because the console codec could not encode a character in a generated answer. The fix was to write records as UTF-8 explicitly and to stop printing predictions to the console; the crash is a reminder that a long unattended run needs I/O that cannot fail on data.

Compressor context limit. Phi-2 cannot hold a 15k-token document, which forced 1,400-token chunked compression and turned Part 3 into a partially different algorithm. This limitation became a research question and produced the Part 4 experiment.

Out-of-memory on Colab. The T4 nodes forced chunked prefill, a halved budget retry path, and a resumable design; without the JSONL checkpoints, several runs would have had to start over.

Environment conflict for Part 4. The 4-bit Qwen compressor required a different software stack that could not be installed on the Windows host that ran the other parts, so Part 4 ran on Colab. This is the reason it has no same-hardware latency measurement, and it also caused the Windows commit-limit failure that blocked running it locally.

Heterogeneous hardware. Four machines produced the four parts, which cost us an extra 30-minute rerun before any latency claim could be made.

4.4 Limitations

- One dataset (`multifieldqa_en`, 150 examples), one target model (3B), one head fraction, two ratios: the ranking is not established beyond this configuration.
- F1 is lexical overlap, not semantic correctness; EM is near zero for sentence-length gold answers and is used only as a sanity check.
- Generation is capped at 64 tokens, which penalises long gold answers.
- Part 3 is a chunked reproduction of LLMLingua, and Part 4 is a multi-factor variant; neither is the paper's exact configuration.
- Part 4 has no same-hardware latency measurement, and its accuracy gain is confounded.
- The position analysis relies on fuzzy matching for 87 of 150 answers and is correlational.
- Greedy decoding is not bit-exact across GPUs, so small differences between runs are noise.

4.5 Future Work

The most direct extension is to repeat the position analysis for the compressed settings, so that the failure modes of Case 3 and Case 4 can be counted rather than illustrated: does compression shift errors towards questions whose answer is a date, a number, or a relation between two entities? Second, the confounding in Part 4 should be removed by an ablation that changes one factor at a time (de-chunking only, then question conditioning only) on a fixed compressor size. Third, the latency result should be completed by running all four parts, including Part 4, on one GPU with one environment, and by measuring time-to-first-token in addition to total time. Fourth, the study should be extended to a larger target model and to benchmarks with uniformly distributed evidence, where the positional baseline is weakest; if the gap shrinks there, the conclusion of this report is a statement about small models and early-evidence datasets rather than about LLMLingua.

5 Use of Generative AI

Generative AI tools were used for literature organisation, for drafting and editing prose, for LaTeX formatting and figure layout, and for code assistance and debugging. They were not used to generate experimental results. Every number in this report was recomputed from the committed result JSON files by a script, and the recomputation was cross-checked against the per-part READMEs before writing; where the two disagreed, the JSON was treated as authoritative. All code, experimental configurations and reported results were reviewed and verified by the project members before submission, and the qualitative cases are produced by a committed selection script rather than written by hand.

References

Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2024. LongBench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3119–3137.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang,

and Lili Qiu. 2023. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 13358–13376.

Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2024. LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1658–1677.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. 2019. Natural Questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466.

Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. 2023. Compressing context to enhance inference efficiency of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 6342–6353.

Zongqian Li, Yinhong Liu, Yixuan Su, and Nigel Collier. 2025. Prompt compression for large language models: A survey. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 5807–5823.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.